

Professionnals I.T. Projects



Objet : Documentation technique - Client Lourd Caballio (Java)

Auteur : Thomas COSTELET, Adrien RUSAOUEN, Jassim
HAMDOUNE

Destinataire : Public

Date : Juin 2026

1. ENVIRONNEMENT TECHNOLOGIQUE

L'application de bureau Caballio a été développée pour répondre aux besoins d'administration centralisée des centres équestres.

- **Langage** : Java (JDK 17+)
- **Interface Graphique** : Bibliothèque Java Swing & AWT
- **Base de données** : MySQL (Serveur distant Linux)
- **Outil de versioning** : Git / GitHub
- **IDE (Environnement de développement)** : Eclipse

2. ARCHITECTURE LOGICIELLE (MVC & DAO)

Afin de garantir un code évolutif, maintenable et sécurisé, l'application respecte de manière stricte le patron de conception **MVC (Modèle-Vue-Contrôleur)**, renforcé par le design pattern **DAO (Data Access Object)**.

- **Le Modèle (DAO)** : Les accès à la base de données sont totalement isolés de l'interface graphique. Chaque entité (Centre, Cheval, Client, Équipement) possède sa propre classe dédiée (ex: ChevalDAO.java). Ces classes contiennent exclusivement les requêtes SQL préparées (PreparedStatement), ce qui protège l'application contre les failles d'injection SQL.
- **La Vue** : L'interface utilisateur utilise le composant CardLayout au sein de la VuePrincipale. Ce gestionnaire permet de superposer les différents panneaux

d'administration dans une même fenêtre, offrant une navigation fluide sans rechargement. Les formulaires d'ajout et de modification exploitent des JDialog modaux pour bloquer l'interface parente durant la saisie.

- **Le Contrôleur :** Les classes métiers (ex: Cheval.java, Centre.java) modélisent les objets manipulés et assurent la transition de données entre la base MySQL et l'affichage (notamment via l'objet DefaultTableModel qui met à jour dynamiquement les composants JTable de l'interface).

3. DÉPLOIEMENT ET SÉCURITÉ DE LA CONNEXION

L'application communique directement avec le serveur MySQL central.

- **Fichier de configuration :** Les identifiants de connexion JDBC (URL, Utilisateur, Mot de passe) ne sont pas codés en dur. Ils sont externalisés dans un fichier config.properties lu par la classe BDD.java. Cela permet le déploiement sur différents environnements sans recompilation du code.
- **Sécurité des mots de passe :** L'authentification repose sur un hachage asymétrique. Le mot de passe saisi par l'administrateur est vérifié via l'algorithme BCrypt (bibliothèque jBCrypt intégrée au code source) qui intègre un grain de sel dynamique avant d'être comparé aux enregistrements.

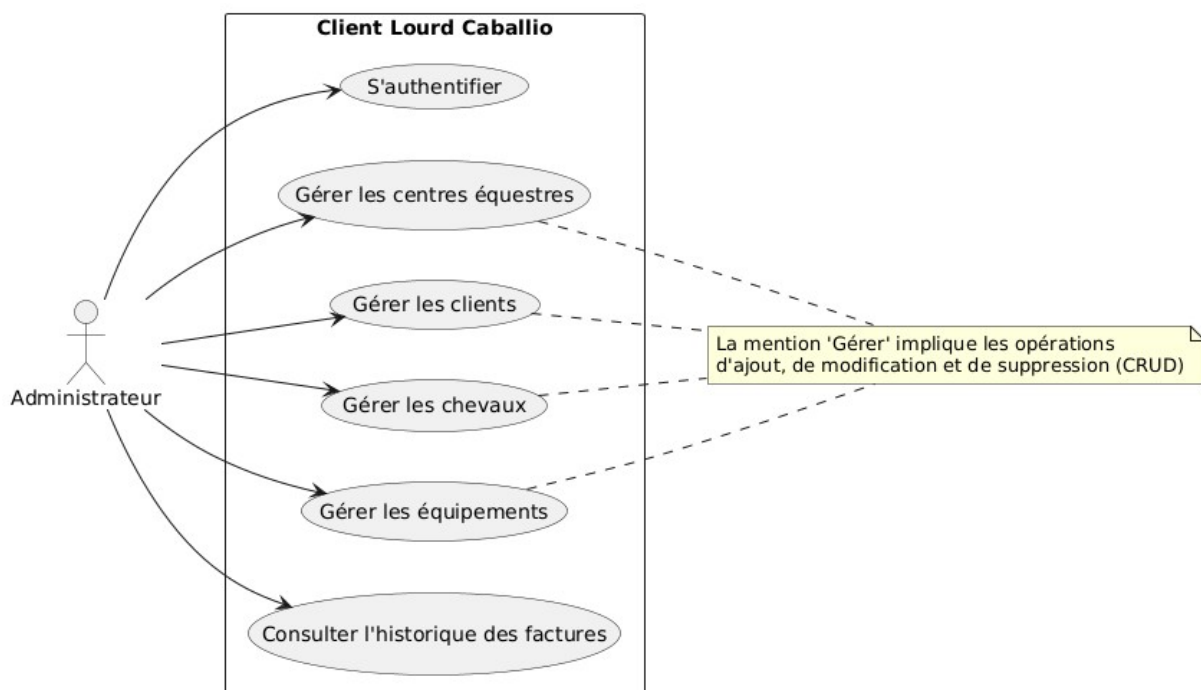
4. STRUCTURE DE LA BASE DE DONNÉES

L'application s'appuie sur la base de données relationnelle MySQL partagée. L'intégrité des données est assurée directement par le SGBD via :

- **Des contraintes de clés étrangères (FK) :** Le code Java ne gère pas les liaisons physiques ; la base impose qu'un équipement soit obligatoirement rattaché à un centre et à un client existants.
- **Des Triggers (Déclencheurs) :** La base intègre des règles de gestion automatisées que le client lourd sollicite implicitement. Par exemple, un trigger bloque techniquement l'insertion d'un cheval si celui-ci a moins de 5 ans. Un autre trigger empêche la création d'un client si son adresse e-mail est déjà utilisée dans le système.

5. DIAGRAMME DE CAS D'UTILISATION (DCU)

Le diagramme ci-dessous modélise les interactions possibles entre l'acteur (l'Administrateur) et le système (le Client Lourd).



6. MODÈLE CONCEPTUEL ET RELATIONNEL DES DONNÉES (MCD)

